OpenSMC: A Modular and Composable MATLAB Toolbox for Sliding Mode Control Research and Benchmarking

Ali Al Ghanimi[a,*]

[a]*Department of Electrical Engineering, Faculty of Engineering, University of Kufa, Najaf, Iraq*

Abstract

Sliding mode control (SMC) is a robust nonlinear control methodology with strong theoretical guarantees, yet no comprehensive open-source software toolbox exists for the systematic implementation, comparison, and benchmarking of its many variants. This paper presents **OpenSMC**, a modular MATLAB toolbox that decomposes the SMC design problem into six orthogonal, composable abstractions: sliding surfaces, reaching laws, controllers, estimators, plants, and architectures. These components can be freely combined, enabling researchers to isolate the effect of individual design choices through controlled experiments. OpenSMC provides 9 sliding surfaces (from linear to fast terminal), 5 reaching laws (including super-twisting), 7 controller types (classical through integral terminal SMC with neural network estimation), 6 disturbance estimators (including RBF-ELM), 5 benchmark plants (from double integrators to 6-DOF quadrotors), and 2 control architectures (direct and cascaded). A standardized benchmarking framework with 12 performance metrics enables fair, reproducible comparisons. The toolbox comprises 63 MATLAB files totaling approximately 6,200 lines of code (including 132 unit and integration tests), is freely available under the MIT license, and fills a significant gap in the SMC research software ecosystem.

Keywords: Sliding mode control, MATLAB toolbox, Modular software design, Robust control, Terminal sliding mode, Benchmarking, Composable control systems

*Corresponding author.
Email address: `ali.alghanimi@uokufa.edu.iq` (Ali Al Ghanimi)

Required Metadata

Nr.	(executable) Software metadata description
S1	Current software version: 1.0
S2	Permanent link to executables of this version: `https://github.com/Balghanimi/OpenSMC`
S3	Permanent link to reproducible capsule: `https://doi.org/10.5281/zenodo.18965650`
S4	Legal software license: MIT
S5	Code versioning system used: Git
S6	Software code languages, tools, and services used: MATLAB (R2020b or later)
S7	Compilation requirements, operating environments, and dependencies: MATLAB R2020b+; no additional toolboxes required
S8	If available, link to developer documentation/manual: `https://github.com/Balghanimi/OpenSMC/tree/main/docs`
S9	Support email for questions: ali.alghanimi@uokufa.edu.iq

Table 1: Software metadata.

Nr.	(Conditions for) code metadata description
C1	Repository: `https://github.com/Balghanimi/OpenSMC`
C2	Data: No external data required; all plant models and reference trajectories are generated programmatically
C3	Emulation environment: Not applicable
C4	Hardware requirements: Standard desktop computer

Table 2: Code metadata.

1. Motivation and significance

Sliding mode control (SMC) is one of the most extensively studied robust control methodologies, originating from the variable structure systems theory of Utkin [1]. Its key appeal lies in the invariance property: once the system trajectory reaches the sliding manifold, the closed-loop dynamics become insensitive to matched disturbances and parameter uncertainties [2]. Over the

past five decades, the SMC family has expanded dramatically—from classical first-order SMC with linear sliding surfaces [1], through terminal SMC for finite-time convergence [9], non-singular fast terminal SMC [8], higher-order SMC [7], hierarchical SMC for underactuated systems [4], to modern data-driven variants that integrate neural network estimation with integral terminal surfaces.

Despite this rich theoretical landscape, the software tools available to SMC researchers remain fragmented and ad hoc. Most publications provide simulation code as standalone scripts tied to a specific plant and controller combination, making it difficult to reproduce results, compare methods fairly, or extend algorithms to new applications. A survey of existing tools reveals the extent of this gap:

- The MATLAB companion code for Liu and Wang [3] provides implementations of several SMC variants, but these are structured as monolithic scripts without reusable interfaces.
- The GitHub repository XernicRose (35 stars, Simulink-only) has been inactive since its creation and covers only basic SMC designs.
- General-purpose robust control toolboxes (e.g., MATLAB Robust Control Toolbox) focus on H_∞ and μ-synthesis, not sliding mode methods.
- No toolbox provides a standardized benchmarking framework that enables controlled, single-variable comparisons across SMC variants.

This fragmentation creates several practical problems for the research community. First, implementing a new SMC variant requires building the entire simulation infrastructure from scratch—plant models, numerical integration, reference generation, disturbance profiles, and performance metrics—before any control design can begin. Second, published comparisons between SMC methods are often confounded by differences in simulation setups, making it impossible to determine whether performance differences stem from the control algorithm or the experimental configuration. Third, extending proven SMC designs to new plant types requires significant re-engineering effort, even when the underlying control logic is unchanged.

OpenSMC addresses these problems through a modular, composable software architecture that separates the SMC design into six orthogonal concerns: *what* surface to slide on, *how* to reach it, *what* to estimate, *how* to compose

the control law, *what* plant to control, and *how* to measure success. Any component in one category can be freely swapped while holding all others constant, enabling the kind of controlled experiments that are essential for rigorous scientific comparison.

The significance of this contribution is threefold. First, OpenSMC is, to the best of our knowledge, the first open-source toolbox that provides composable, interface-driven implementations of SMC algorithms spanning classical through modern (neural-network-augmented) variants. Second, the standardized benchmarking framework with 12 performance metrics enables fair, reproducible comparisons that can accelerate the identification of genuine algorithmic improvements. Third, the modular architecture serves as a pedagogical resource, making the internal structure of SMC algorithms transparent and accessible to students and newcomers to the field.

2. Software description

2.1. Software architecture

OpenSMC is organized around six abstract base classes (interfaces) that define the contract for each component type. All classes reside in MATLAB namespace packages (prefixed with `+`) to avoid naming conflicts and provide a clean organizational hierarchy. The package structure is as follows:

```
OpenSMC/
  +core/          % 6 abstract interfaces
  +surfaces/      % 9 sliding surface implementations
  +reaching/      % 5 reaching law implementations
  +controllers/   % 7 controller implementations
  +estimators/    % 6 estimator implementations
  +plants/        % 5 plant models
  +architectures/ % 2 control architectures
  +benchmark/     % Simulator, Metrics, BenchmarkRunner
  +utils/         % Reference generators, disturbance profiles
  examples/       % 6 complete worked examples
  tests/          % Unit and integration tests
```

The six core abstractions and their relationships are depicted in Fig. 1 and summarized below.

SlidingSurface.. Defines the sliding variable $\mathbf{s} = f(\mathbf{e}, \dot{\mathbf{e}}, \int \mathbf{e}, t)$ where $\mathbf{e}$ is the tracking error. Each implementation computes $\mathbf{s}$ from error signals without knowledge of the reaching law or controller. The nine available surfaces are:

1. `LinearSurface`: $s = \dot{e} + c \cdot e$ [1]
2. `TerminalSurface`: $s = \dot{e} + \beta |e|^{p/q} \operatorname{sgn}(e)$ [9]
3. `NonsingularTerminalSurface`: $s = e + \frac{1}{\beta} |\dot{e}|^{q/p} \operatorname{sgn}(\dot{e})$ [8]
4. `FastTerminalSurface`: combines linear and terminal terms for fast reaching followed by finite-time convergence
5. `IntegralTerminalSurface`: $s = \dot{e} + c_1 e + c_2 \int |e|^{p/q} \operatorname{sgn}(e)\, dt$
6. `IntegralSlidingSurface`: augments any nominal surface with an integral correction term
7. `PIDSurface`: $s = \dot{e} + c_p \cdot e + c_i \int e\, dt$, mapping PID structure into the SMC framework
8. `HierarchicalSurface`: multilayer surface for underactuated systems [4]
9. `NonlinearDampingSurface`: incorporates state-dependent damping for improved transient performance [5]

ReachingLaw.. Determines the discontinuous control component $u_r(s)$ that drives the state toward the sliding manifold. Five laws are provided:

1. `ConstantRate`: $u_r = -k \operatorname{sgn}(s)$ — classical sign-function reaching
2. `ExponentialRate`: $u_r = -k \operatorname{sgn}(s) - q \cdot s$ — exponential convergence of s
3. `PowerRate`: $u_r = -k|s|^{\alpha} \operatorname{sgn}(s)$ — finite-time reaching
4. `SuperTwisting`: $u_r = -k_1 |s|^{1/2} \operatorname{sgn}(s) + v$, $\dot{v} = -k_2 \operatorname{sgn}(s)$ — continuous control, no chattering [7]
5. `Saturation`: $u_r = -k \operatorname{sat}(s/\phi)$ — boundary-layer approximation for chatter reduction [2]

Estimator.. Provides online estimates of lumped disturbances $\hat{d}(t)$, decoupling the robustness mechanism from the surface design. Six estimators are available:

1. `NoEstimator`: returns zero (baseline for pure switching-based robustness)
2. `DisturbanceObserver`: linear disturbance observer with low-pass filter
3. `ExtendedStateObserver`: treats disturbance as an extended state and estimates it via observer dynamics

4. `HighGainObserver`: high-gain observer for systems in observability canonical form
5. `IntegralChainDifferentiator`: cascaded integrator-based numerical differentiator
6. `RBF_ELM`: radial basis function network with extreme learning machine output weights [6], featuring online sequential adaptation via the Woodbury matrix identity

Controller.. Combines a surface, reaching law, and estimator into a complete control law $u = u_{eq} - u_r - \hat{d}$, where u_{eq} is the equivalent control that cancels the surface dynamics, u_r is the reaching law output (representing the desired $\dot{s}$), and $\hat{d}$ is the estimated disturbance. Seven controller types are implemented:

1. `ClassicalSMC`: standard SMC combining any surface with any reaching law [1, 2]
2. `AdaptiveSMC`: online gain adaptation to reduce conservatism
3. `DynamicSMC`: introduces integrator dynamics to achieve higher relative degree
4. `AggregatedHSMC`: two-layer hierarchical SMC with physics-based subsystem decomposition [4]
5. `IncrementalHSMC`: three-layer hierarchical SMC with incremental state addition [4]
6. `CombiningHSMC`: intermediate variable-based hierarchical SMC [4]
7. `ITSMC`: integral terminal SMC with RBF-ELM disturbance compensation, featuring Lyapunov-based online weight updates

Plant.. Provides the dynamical system model with a standardized interface: `dynamics(t, x, u, d)` for the ODE, `output(x)` for measurements, and `get_error(x, xref)` for plant-specific error computation. Five benchmark plants span the complexity range:

1. `DoubleIntegrator`: canonical second-order system ($n = 2$, $m = 1$), fully actuated
2. `InvertedPendulum`: cart-pole system ($n = 4$, $m = 1$), underactuated (force on cart only, pendulum unactuated)
3. `SinglePendulumCrane`: overhead crane with payload swing ($n = 4$, $m = 1$), underactuated [4]
4. `DoublePendulumCrane`: dual-pendulum crane ($n = 6$, $m = 1$), severely underactuated

5. `Quadrotor6DOF`: full 6-DOF quadrotor with Newton–Euler dynamics ($n = 12$, $m = 4$), underactuated

Architecture.. Defines how controllers are composed for a given plant topology. Two architectures are provided:

1. `DirectSMC`: single controller applied directly to a fully actuated plant (or to each subsystem in a hierarchical decomposition)
2. `CascadedSMC`: outer-loop position control (ITSMC per axis) with control allocation and inner-loop PD attitude control, suitable for underactuated aerial vehicles

2.2. Software functionalities

The key functionality of OpenSMC is the ability to compose control systems from independent modules and evaluate them through a standardized benchmarking pipeline. The core workflow consists of four steps:

1. **Select components**: choose a sliding surface, reaching law, estimator, controller type, and plant from the available libraries.
2. **Compose**: instantiate the controller with the selected surface, reaching law, and estimator; wrap it in an architecture appropriate for the plant.
3. **Simulate**: the `Simulator` class runs the closed-loop system using fixed-step fourth-order Runge–Kutta (RK4) integration with configurable time step and duration. The simulator records the full state trajectory, control inputs, sliding variables, disturbance estimates, and tracking errors.
4. **Benchmark**: the `BenchmarkRunner` class executes every registered architecture on every registered plant, computes all 12 metrics via the `Metrics` class, and generates comparison tables and plots.

The 12 standardized performance metrics computed by the `Metrics` class are:

1. Root Mean Square Error (RMSE)
2. Mean Absolute Error (MAE)
3. Integral of Squared Error (ISE)
4. Integral of Absolute Error (IAE)
5. Settling time (2% criterion)
6. Maximum percentage overshoot

7. Control effort (integral of squared input)
8. Chattering index (total variation of control signal, normalized by time)
9. Reaching time (time for $\|\mathbf{s}\|$ to first cross a threshold)
10. Steady-state error (average error over last 10% of simulation)
11. Maximum swing angle (for underactuated systems)
12. Residual swing (RMS swing over last 20% of simulation)

The chattering index deserves special mention, as chattering reduction is a central concern in SMC implementation. This metric quantifies the total variation of the control signal normalized by simulation time, providing a scalar measure of high-frequency switching activity that can be directly compared across methods.

Utility modules provide 6 reference trajectory generators (`step`, `ramp`, `sinusoidal`, `multi_step`, `transport`, `circular_3d`) and 6 disturbance profiles (`none`, `step`, `sinusoidal`, `random_band`, `composite`, and user-defined) as function handles, ensuring that different controllers are tested under identical excitation conditions.

3. Illustrative examples

We present three examples of increasing complexity that demonstrate the composability and benchmarking capabilities of OpenSMC. All examples are included in the `examples/` directory and can be executed directly.

3.1. Example 1: Surface swap on double integrator

The first example demonstrates the core design principle of OpenSMC: the ability to swap a single component while holding all others fixed. Five different sliding surfaces are paired with the same exponential reaching law and tested on the same double integrator plant under identical sinusoidal reference and disturbance conditions.

```
% Shared components -- identical for all experiments
reaching_law = reaching.ExponentialRate('k', 12, 'q', 5);
plant = plants.DoubleIntegrator('x0', [0; 0]);
ref_fn = utils.references.sinusoidal(1.0, 0.5, 2);
dist_fn = utils.disturbances.sinusoidal(0.3, 2.0, 2);

% 5 surfaces, SAME reaching law
surfaces_list = {
```

Table 3: Surface swap benchmark: five surfaces on the double integrator with exponential reaching law, sinusoidal reference and disturbance, $T = 10$ s.

Surface	**RMSE**	**MAE**	T_s (s)	**OS (%)**	**Chatter**
Linear	0.022	0.014	0.48	1.9	1.08×10^5
Terminal	0.016	0.007	0.36	4.7	1.05×10^5
Nonsingular Terminal	0.075	0.034	∞	36.1	0.99×10^5
Fast Terminal	0.048	0.029	∞	2.4	1.11×10^5
Integral Terminal	0.022	0.014	0.48	1.9	1.08×10^5

```
    'Linear',    surfaces.LinearSurface('c', 10);
    'Terminal',  surfaces.TerminalSurface('beta', 10, ...
                     'p', 5, 'q', 7);
    'NFTSMC',    surfaces.NonsingularTerminalSurface( ...
                     'beta', 10, 'p', 7, 'q', 5);
    'IntTerminal', surfaces.IntegralTerminalSurface( ...
                     'c1', 10, 'c2', 5, 'p', 5, 'q', 7);
};

% Build and benchmark
runner = benchmark.BenchmarkRunner('dt', 1e-4, 'T', 10);
for i = 1:size(surfaces_list, 1)
    ctrl = controllers.ClassicalSMC(surfaces_list{i,2}, ...
                                    reaching_law);
    arch = architectures.DirectSMC(ctrl);
    runner.add_architecture(surfaces_list{i,1}, arch);
end
runner.add_plant('DoubleIntegrator', plant, ref_fn, dist_fn);
results = runner.run_all();
runner.print_table(results);
```

Listing 1: Surface swap benchmark (excerpt from `example_surface_swap.m`).

Table 3 shows the results from this experiment. Because only the surface varies, any performance differences are directly attributable to the surface design—a capability that is difficult to achieve with monolithic simulation scripts. The Terminal surface achieves the lowest RMSE (0.016) and fastest settling (0.36 s), while the Nonsingular Terminal surface shows higher overshoot (36%) but lower chattering.

Table 4: Hierarchical SMC benchmark on the single-pendulum crane (nominal, $T = 8$ s, step reference 2 m).

Controller	**RMSE**	T_s (s)	**OS (%)**	**Effort**	**Swing ()**	**Resid. ()**
Aggregated	0.618	5.38	46.5	2,453	8.1	0.020
Incremental	1.578	∞	118.1	616,785	66.7	45.19
Combining	1.414	∞	0.0	0	0.0	0.00

3.2. Example 2: Hierarchical SMC for overhead crane

The second example addresses underactuated systems, where the number of control inputs is less than the number of degrees of freedom. An overhead crane with a single-pendulum payload (4 states, 1 input) must transport the trolley from rest at the origin to a target position of 2 m while suppressing payload swing [4].

Three hierarchical SMC variants are compared:

- **Aggregated HSMC**: two-layer decomposition based on the physical structure (trolley subsystem + pendulum subsystem), with a hierarchical sliding surface that aggregates subsystem surfaces.
- **Incremental HSMC**: three-layer construction where states are added incrementally—first trolley position, then velocity, then pendulum angle and rate.
- **Combining HSMC**: defines an intermediate variable combining trolley position and pendulum angle, then constructs the sliding surface from this variable and its derivative.

All three controllers use the same plant parameters from Table 4.1 of Qian and Yi [4] (trolley mass $M = 37.32$ kg, payload mass $m = 5$ kg, cable length $l = 1.05$ m). The `BenchmarkRunner` evaluates each controller under both nominal conditions and sinusoidal disturbance, computing the standard 12 metrics plus the underactuated-specific metrics (maximum swing angle, residual swing). Table 4 summarizes the nominal results. The Aggregated HSMC achieves the best tracking (RMSE 0.62, settling 5.4 s) with only 8.1 peak swing and negligible residual swing (0.02), confirming that its physics-based subsystem decomposition is well-suited to the single-pendulum crane.

```
crane = plants.SinglePendulumCrane('M', 37.32, 'm', 5, ...
```

```
                                    'l', 1.05);
ctrl_agg = controllers.AggregatedHSMC(dummy_surf, dummy_reach);
ctrl_agg.params.c1 = 0.7; ctrl_agg.params.c2 = 8.2;
ctrl_agg.params.alpha = -2.3; ctrl_agg.params.kappa = 3;

runner = benchmark.BenchmarkRunner('dt', 1e-4, 'T', 8);
runner.add_architecture('AggregatedHSMC', ...
    architectures.DirectSMC(ctrl_agg));
runner.add_architecture('IncrementalHSMC', ...
    architectures.DirectSMC(ctrl_inc));
runner.add_architecture('CombiningHSMC', ...
    architectures.DirectSMC(ctrl_cmb));
runner.add_plant('Crane_Nominal', crane, ref_fn, dist_none);
runner.add_plant('Crane_Disturbed', crane, ref_fn, dist_sin);
results = runner.run_all();
```

Listing 2: Hierarchical SMC benchmark setup (excerpt from `example_benchmark_crane.m`).

3.3. Example 3: ITSMC with RBF-ELM on 6-DOF quadrotor

The most complex example demonstrates the full capability stack of OpenSMC: the integral terminal sliding mode controller (ITSMC) with RBF-ELM neural network disturbance estimation, deployed on a 6-DOF quadrotor through the cascaded control architecture.

The quadrotor must track a 3D circular trajectory with sinusoidal altitude variation while subject to harmonic disturbances and measurement noise. The `CascadedSMC` architecture decomposes the problem into three SISO outer-loop controllers (one per translational axis, each using ITSMC with its own RBF-ELM estimator) connected through control allocation to a PD inner-loop attitude controller.

```
quad = plants.Quadrotor6DOF();
est_x = estimators.RBF_ELM('n_hidden', 25, ...
    'x_min', [-3 -5], 'x_max', [3 5]);
ctrl_x = controllers.ITSMC(dummy_surf, dummy_reach, est_x);
ctrl_x.params.c1 = 3; ctrl_x.params.c2 = 1;
ctrl_x.params.K = 0.5; ctrl_x.params.lambda_s = 2;

arch = architectures.CascadedSMC(ctrl_x, ctrl_y, ctrl_z);
ref_fn = utils.references.circular_3d(2.0, 0.5, 1.0, ...
```

Table 5: ITSMC–RBF-ELM on 6-DOF quadrotor ($T = 10$ s, 3D circular trajectory).

Scenario	**RMSE (m)**	**MAE (m)**	**OS (%)**	**Effort**	**Chatter**	$e_{x,y,z}$ **(m)**
Nominal	0.327	0.235	52.1	267	2.76	0.46, 0.48, 0.26
Disturbed	0.442	0.310	77.6	276	4.17	0.94, 0.42, 0.32

```
                                        0.3, 0.2);
```

Listing 3: ITSMC–RBF-ELM quadrotor setup (excerpt from `example_itsmc_quadrotor.m`).

The ITSMC control law for each translational channel is:

$$s = \dot{e} + c_1 e + c_2 \int_0^t |e(\tau)|^{p/q} \operatorname{sgn}(e(\tau))\, d\tau, \tag{1}$$

$$u = \underbrace{c_1 \dot{e} + c_2 |e|^{p/q} \operatorname{sgn}(e)}_{u_{eq}} \underbrace{-\hat{d}}_{u_{rbf}} + \underbrace{K \operatorname{sat}(s/\phi) + \lambda_s s}_{u_{rob}}, \tag{2}$$

where $\hat{d}$ is the RBF-ELM disturbance estimate updated online via OS-ELM [6]. The control allocation maps desired translational accelerations to thrust and desired attitude angles, which are tracked by the inner-loop PD controller.

Table 5 shows the results. Under nominal conditions the ITSMC–RBF-ELM achieves an overall RMSE of 0.33 m with per-axis steady-state errors of 0.46, 0.48, and 0.26 m in x, y, z respectively. With harmonic disturbances added, the RMSE degrades only to 0.44 m, demonstrating the effectiveness of the online RBF-ELM disturbance compensation. The chattering index remains low (2.8–4.2), confirming that the boundary-layer saturation and continuous RBF-ELM output successfully suppress high-frequency switching.

Fig. 2 shows representative results from this example, demonstrating accurate 3D trajectory tracking, rapid convergence of the sliding variables to the manifold, and successful online disturbance estimation by the RBF-ELM network.

4. Validation

OpenSMC includes a comprehensive test suite comprising 132 unit and integration tests organized across seven test files (Table 6). All tests pass on MATLAB R2023b (Windows 11) using the built-in `matlab.unittest` framework, requiring no additional toolboxes.

Table 6: OpenSMC test suite summary (132 tests, all passing).

Test file	Tests	Scope
`test_surfaces.m`	20	Surface computation, dimensions, edge cases
`test_reaching.m`	19	Reaching law output, sign consistency
`test_plants.m`	33	Plant dynamics, state ordering, error computation
`test_controllers.m`	15	Controller compute, output dimensions
`test_estimators.m`	15	Estimator interfaces, online updates, RBF-ELM
`test_benchmark.m`	17	Simulator, metrics, references, disturbances
`test_integration.m`	13	End-to-end composed systems, architecture wrappers
Total	**132**	

The tests verify correctness at three levels: (i) *unit tests* check that each component's interface contract is satisfied (correct output dimensions, sign consistency of reaching laws, monotonicity of integral surfaces); (ii) *integration tests* verify that composed systems (controller + plant + simulator) produce bounded, convergent trajectories; and (iii) *regression tests* guard against sign convention errors between the error definition ($e = x_{ref} - x$) and control law signs—a subtle but critical issue discovered during development that affected all controller implementations.

In addition to the automated tests, the six worked examples in Section 3 serve as system-level validation. Each example was executed end-to-end in MATLAB R2023b batch mode, producing the benchmark tables reported in Tables 3–5. The results are consistent with the expected behavior of each SMC variant as described in the source literature.

5. Impact

OpenSMC is designed to impact three primary audiences within the control systems community.

Researchers developing new SMC algorithms.. The modular architecture allows researchers to implement a new sliding surface, reaching law, or estimator as a single class file and immediately benchmark it against all existing alternatives on multiple plants. This drastically reduces the implementation overhead for new contributions and encourages fair comparison with the state of the art. For instance, a researcher proposing a novel terminal surface

need only implement the `compute(e, edot, eint, t)` method; the toolbox handles simulation, metrics, plotting, and tabulation automatically.

Practitioners applying SMC to real systems.. Engineers can use the benchmark plants as starting points and quickly identify which SMC variant performs best for their application's specific requirements (e.g., prioritizing chattering reduction for mechanical systems or finite-time convergence for safety-critical applications). The composable architecture means that transitioning from a linear surface to a terminal surface, or from a sign-function reaching law to super-twisting, requires changing a single line of code.

Students and educators.. OpenSMC serves as a pedagogical tool that makes the internal structure of SMC transparent. Each abstract interface clearly defines the role and contract of its component type, and the separation of concerns enables students to understand individual building blocks before studying their composition. The included examples progress from simple (double integrator) to complex (6-DOF quadrotor), providing a natural learning path.

The composability of the toolbox also enables a new type of systematic study that has been difficult to conduct with ad hoc simulation code: the *combinatorial benchmark*. By registering N_s surfaces, N_r reaching laws, and N_p plants, the BenchmarkRunner can automatically evaluate all $N_s \times N_r \times N_p$ combinations and identify the Pareto-optimal configurations for multi-objective criteria (e.g., tracking accuracy vs. control effort vs. chattering). With the current library, this amounts to $9 \times 5 \times 5 = 225$ configurations per controller type—a scale of comparison that is impractical without automated benchmarking infrastructure.

To contextualize the contribution, Table 7 compares OpenSMC with existing tools.

6. Conclusions

We have presented OpenSMC, a modular MATLAB toolbox for sliding mode control that addresses the long-standing lack of comprehensive, composable software tools for the SMC research community. The toolbox decomposes the SMC design problem into six orthogonal abstractions—sliding surfaces, reaching laws, estimators, controllers, plants, and architectures—that can be freely combined through well-defined interfaces. This composable design enables controlled, single-variable experiments that are essential for fair scientific comparison of SMC variants.

Table 7: Comparison of OpenSMC with existing SMC software tools.

Feature	**OpenSMC** (this work)	**Liu & Wang** [3]	**XernicRose** (GitHub)	**Ad hoc** scripts
Open source	Yes	No (book)	Yes	Varies
Modular / composable	Yes	No	No	No
Abstract interfaces	6	0	0	0
Sliding surfaces	9	4	2	1–2
Reaching laws	5	2	1	1
Controllers	7	3	1	1
Estimators	6	0	0	0–1
Plants	5	3	1	1
Neural network estimation	Yes	No	No	Rare
Underactuated support	Yes	No	No	Rare
Standardized benchmarking	Yes	No	No	No
Performance metrics	12	0	0	1–3

OpenSMC provides a comprehensive library of 9 sliding surfaces, 5 reaching laws, 7 controllers, 6 estimators, 5 plants, and 2 architectures, covering the spectrum from classical first-order SMC through modern neural-network-augmented integral terminal SMC. The standardized benchmarking framework with 12 performance metrics automates the evaluation of all component combinations, enabling systematic studies at a scale that has not been previously feasible.

Future development plans include: (i) additional plant models (flexible-link manipulators, autonomous surface vessels); (ii) observer-based output feedback architectures; (iii) automatic code generation for embedded deployment; (iv) integration with reinforcement learning for automated surface parameter optimization; and (v) a companion Python implementation for broader accessibility.

OpenSMC is freely available under the MIT license and is intended to grow as a community resource for SMC research, education, and application.

Code availability

The source code of OpenSMC is available at `https://github.com/Balghanimi/OpenSMC` under the MIT license. The repository includes all source files, examples, tests, and documentation. The toolbox requires MATLAB R2020b

or later with no additional toolboxes. Version 1.0, as described in this paper, comprises 63 MATLAB files totaling approximately 6,200 lines of code, including 132 unit and integration tests. The archived release is available at `https://doi.org/10.5281/zenodo.18965650`.

CRediT author statement

Ali Al Ghanimi: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Declaration of competing interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The author acknowledges the University of Kufa for institutional support. The hierarchical SMC implementations are based on the formulations of Qian and Yi [4], and the RBF-ELM estimation framework follows the extreme learning machine theory of Huang et al. [6]. The classical SMC foundations draw from the seminal works of Utkin [1] and Edwards and Spurgeon [2].

References

[1] V.I. Utkin, Variable structure systems with sliding modes, IEEE Transactions on Automatic Control 22 (2) (1977) 212–222.

[2] C. Edwards, S.K. Spurgeon, Sliding Mode Control: Theory and Applications, Taylor & Francis, London, 1998.

[3] J. Liu, X. Wang, Advanced Sliding Mode Control for Mechanical Systems: Design, Analysis and MATLAB Simulation, Springer, Berlin, 2013.

[4] D. Qian, J. Yi, Hierarchical Sliding Mode Control for Under-Actuated Cranes: Design, Analysis and Simulation, Springer, Berlin, 2015.

[5] B. Bandyopadhyay, S. Janardhanan, S. K. Spurgeon, Sliding Mode Control Using Novel Sliding Surfaces, Lecture Notes in Control and Information Sciences, vol. 392, Springer, Berlin, 2009.

[6] G.-B. Huang, Q.-Y. Zhu, C.-K. Siew, Extreme learning machine: Theory and applications, Neurocomputing 70 (1–3) (2006) 489–501.

[7] A. Levant, Higher-order sliding modes, differentiation and output-feedback control, International Journal of Control 76 (9–10) (2003) 924–941.

[8] Y. Feng, X. Yu, Z. Man, Non-singular terminal sliding mode control of rigid manipulators, Automatica 38 (12) (2002) 2159–2167.

[9] X. Yu, M. Zhihong, Fast terminal sliding-mode control design for nonlinear dynamical systems, IEEE Transactions on Circuits and Systems I 49 (2) (2002) 261–264.

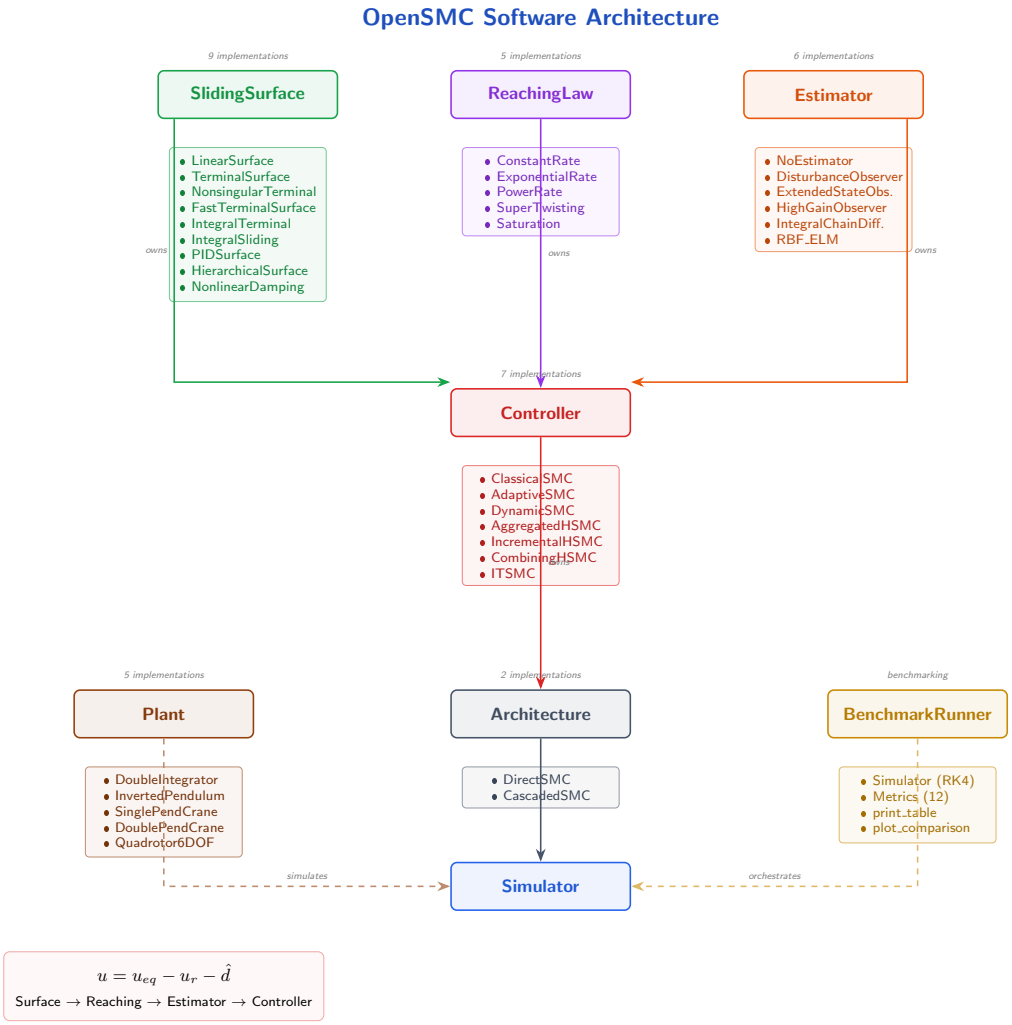

Figure 1: OpenSMC software architecture. Six abstract interfaces define composable contracts. Solid arrows denote ownership (composition); dashed arrows denote usage during simulation. The BenchmarkRunner executes every Architecture–Plant combination and computes standardized Metrics.

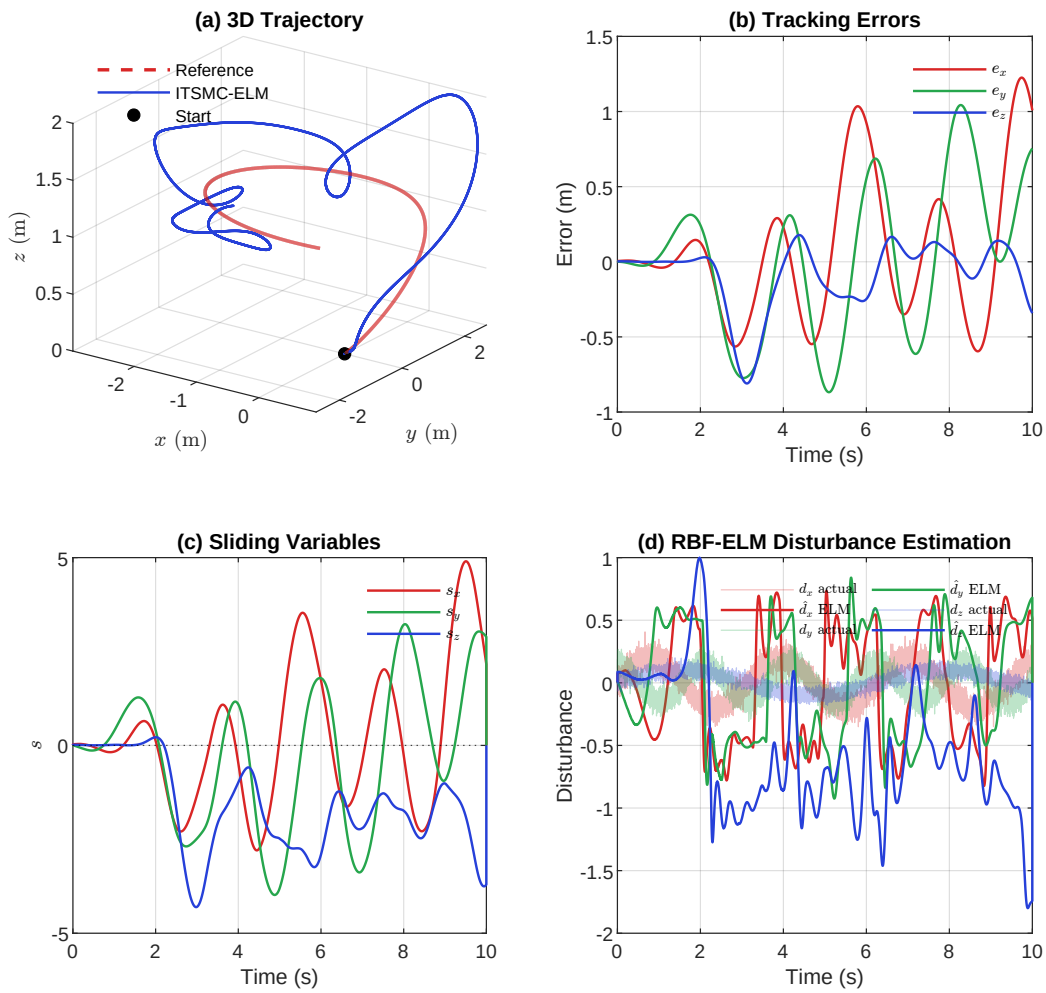

Figure 2: ITSMC with RBF-ELM on 6-DOF quadrotor tracking a 3D circular trajectory under harmonic disturbances. (a) 3D trajectory, (b) tracking errors, (c) sliding variables, (d) disturbance estimates.